



Attention is All you need

1. Introduction

- 기존 시퀀스 변환(sequence transduction) 모델은 주로 **RNN 기반 encoder-decoder 구조**를 사용
 - 특히 LSTM, GRU가 언어 모델링·기계번역에서 SOTA로 자리 잡음
 - 입력 시퀀스를 순차적으로 처리하며 hidden state를 시간축으로 전달
- RNN의 근본적 한계
 - 계산이 **순차적(sequential)**으로 진행되어 병렬화가 어렵다
 - 시퀀스 길이가 길어질수록 학습 속도와 메모리 효율이 급격히 저하
 - long-range dependency 학습이 구조적으로 비효율적
- Attention 메커니즘의 등장
 - 입력·출력 간 장거리 의존성을 직접 연결 가능
 - 거리와 무관하게 중요한 정보를 참조할 수 있음
 - 하지만 대부분의 기존 모델은 **RNN + Attention 구조**
 - attention은 보조적 역할이었음
- 본 논문의 핵심 제안
 - **Recurrence와 Convolution을 완전히 제거**
 - 오직 **Self-Attention 기반 구조만으로 구성된 Transformer 제안**
 - 전역 의존성을 직접 모델링
 - 병렬화 가능 → 학습 속도 대폭 향상
- 주요 실험 결과
 - WMT 2014 English-German 번역에서 28.4 BLEU 달성
 - 기존 최고 성능(앙상블 포함) 대비 2 BLEU 이상 개선
 - English-French에서도 새로운 single-model SOTA 기록

- 8개의 P100 GPU로 12시간 내 학습 가능
- 핵심 메시지
 - **Sequence modeling에서 recurrence는 필수가 아니다**
 - Attention만으로도 더 빠르고 더 좋은 성능 달성 가능
 - Transformer는 이후 다양한 NLP 및 비전 과제로 확장 가능성 제시

2. Background

2.1 Sequence-to-Sequence 모델의 발전

- 기존 sequence transduction 모델은 대부분 **RNN 기반 encoder-decoder 구조**
 - Encoder: 입력 시퀀스를 고정 길이 벡터로 인코딩
 - Decoder: 해당 벡터를 기반으로 출력 시퀀스 생성
 - LSTM, GRU 같은 구조가 long dependency 문제를 완화
 - 이후 **attention mechanism** 도입으로 성능 크게 개선
 - 출력 토큰을 생성할 때 입력 전체를 참조 가능
 - 고정 길이 벡터 병목 문제 해결
-

2.2 Convolution 기반 접근

- RNN의 순차적 계산 한계를 극복하기 위해 CNN 기반 모델 등장
 - 입력을 병렬 처리 가능
 - 계산 효율성 개선
 - 하지만 CNN은:
 - receptive field가 제한적
 - 멀리 떨어진 토큰 간 관계를 학습하려면 깊은 네트워크 필요
 - 결국 long-range dependency modeling이 구조적으로 비효율적
-

2.3 Self-Attention의 개념

- Self-attention은 동일 시퀀스 내부의 토큰 간 관계를 직접 계산
 - 각 위치가 다른 모든 위치를 참조

- 장거리 의존성 모델링이 거리와 무관
 - RNN/CNN과 비교 시 장점:
 - 경로 길이(path length)가 짧음
 - 병렬 계산 가능
 - 장거리 dependency 학습에 유리
-

2.4 연산 복잡도 비교

논문에서는 세 가지 구조를 비교함:

- Self-Attention
- Recurrent layer
- Convolution layer

비교 기준:

1. 한 layer당 계산 복잡도
2. 병렬화 가능성
3. long-range dependency를 연결하는 최소 경로 길이

핵심 결론:

- Self-attention은 sequence 길이에 대해 $O(n^2)$ 복잡도를 가지지만
 - 대부분 NLP task에서는 n 이 hidden dimension보다 작음
- dependency path length가 1
 - RNN은 $O(n)$
 - CNN은 $O(\log n)$ 또는 그 이상
- 따라서 **long dependency modeling**에서 가장 직접적이고 효율적

3. Model Architecture

Transformer는 **encoder-decoder** 구조를 유지하지만,
RNN/CNN 없이 **attention**만으로 구성된 구조이다.

3.1 전체 구조 개요

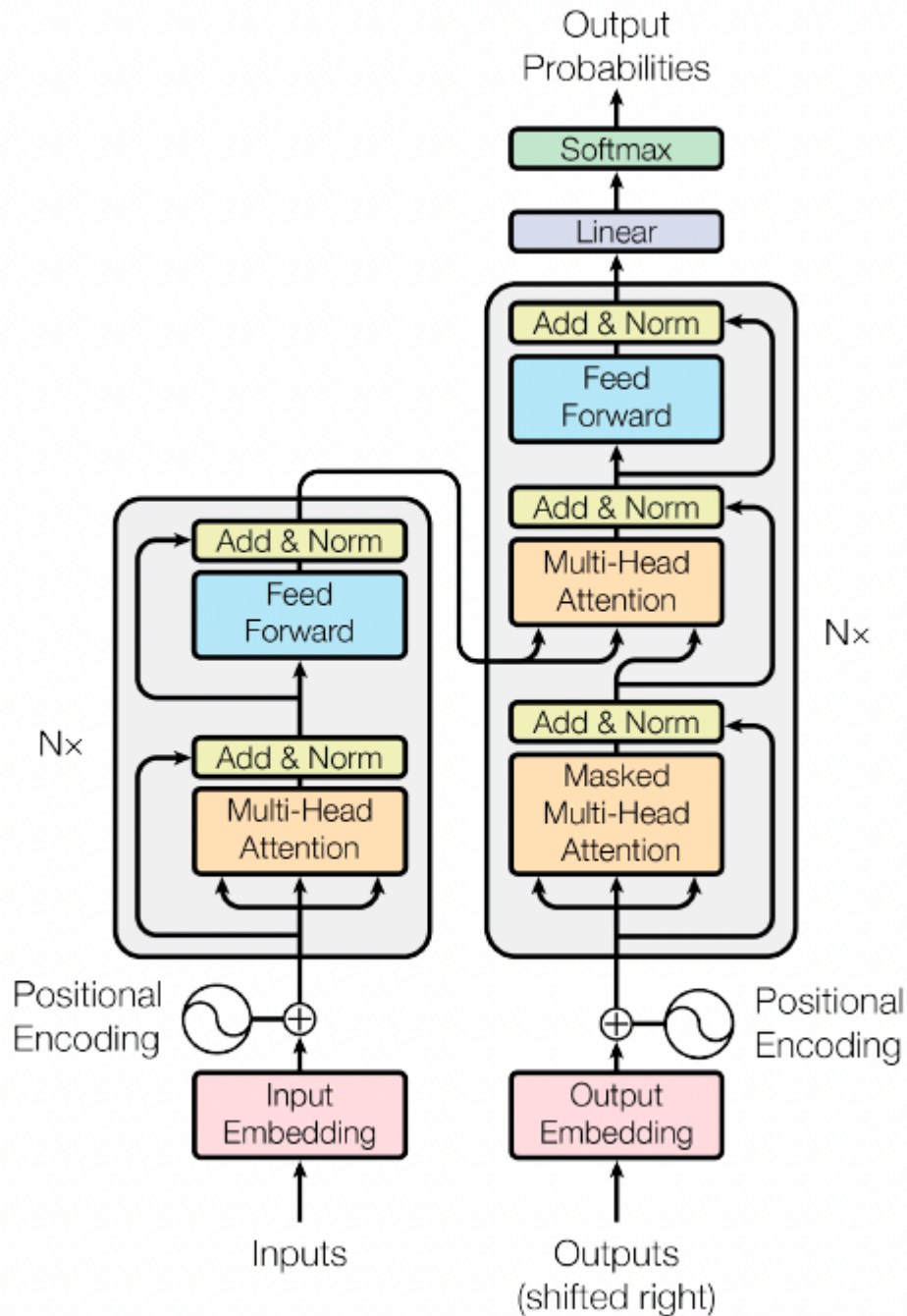


Figure 1: The Transformer - model architecture.

- 기본 구조는 기존 seq2seq와 동일:
 - Encoder: 입력 시퀀스를 표현 벡터로 변환
 - Decoder: 해당 표현을 기반으로 출력 시퀀스 생성

- 차이점:
 - 모든 연산이 **attention + feed-forward**로 구성
 - recurrence 없음
 - convolution 없음
 - 구조 특징:
 - Encoder와 Decoder는 각각 동일한 블록을 여러 층 반복
 - 논문 기본 설정:
 - Encoder 6 layers
 - Decoder 6 layers
 - d_model = 512
 - d_ff = 2048
-

3.2 Encoder

Encoder 구성

Encoder는 동일한 레이어를 N번(기본 6번) 반복.

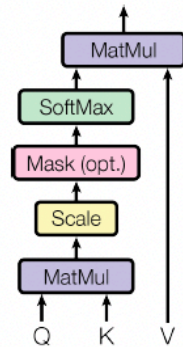
각 레이어는 2개의 sub-layer로 구성:

1. **Multi-Head Self-Attention**
2. **Position-wise Feed Forward Network**

각 sub-layer 뒤에는:

- Residual connection
- Layer Normalization

Scaled Dot-Product Attention



Multi-Head Attention

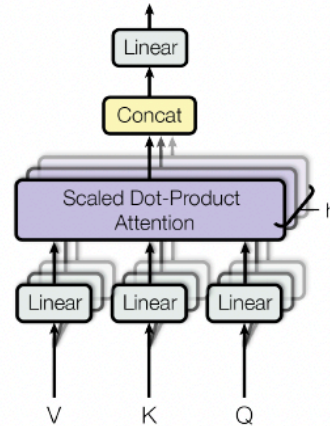


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

(1) Multi-Head Self-Attention

- 입력 시퀀스 내부 토큰 간 관계를 계산
- 각 토큰이 모든 토큰을 참조

Attention 계산 공식:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

- Q: Query
- K: Key
- V: Value

Scaled dot-product attention을 사용

- $\sqrt{d_k}$ 로 나누는 이유:
 - 차원이 커질수록 dot product 값이 커지는 문제 완화

Multi-Head Attention

- 단일 attention을 여러 개 병렬 수행
- 서로 다른 projection space에서 attention 수행

과정:

1. Q, K, V를 각각 h개로 선형 변환

2. 각 head에서 attention 계산
3. 결과 concat
4. 다시 선형 변환

장점:

- 다양한 관계를 동시에 학습
 - 하나의 attention보다 표현력이 풍부
-

(2) Position-wise Feed Forward Network

- 각 position에 독립적으로 적용되는 fully connected layer 2개
- 구조:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- hidden dimension 512 → 2048 → 512
 - 비선형성: ReLU
-

3.3 Decoder

Decoder도 N개(기본 6개) 레이어로 구성.

각 레이어는 3개의 sub-layer:

1. Masked Multi-Head Self-Attention
2. Encoder-Decoder Attention
3. Feed Forward Network

각 sub-layer 뒤:

- Residual connection
 - LayerNorm
-

(1) Masked Self-Attention

- 미래 토큰을 보지 못하도록 mask 적용
 - autoregressive 특성 유지
 - 학습 시에도 causal 구조 유지
-

(2) Encoder-Decoder Attention

- Decoder Query
- Encoder Output → Key, Value

→ 출력 생성 시 입력 시퀀스 참조

3.4 Positional Encoding

Transformer는 recurrence가 없기 때문에

순서 정보(position 정보)가 없다

이를 보완하기 위해 positional encoding을 입력에 더함.

Sinusoidal Positional Encoding

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

특징:

- 학습 파라미터 없음
 - 상대적 위치 정보 추론 가능
 - 길이 extrapolation 가능
-

3.5 Output Layer

- Decoder 출력 → Linear projection
- Softmax → 다음 토큰 확률 분포

4. Why Self-Attention

이 장에서는 Self-Attention을

Recurrent layer와 Convolution layer와 비교한다.

비교 기준은 세 가지이다:

1. 계산 복잡도 (Computational Complexity)
 2. 병렬화 가능성 (Parallelization)
 3. Long-range dependency를 연결하는 경로 길이 (Path Length)
-

4.1 Layer당 계산 복잡도 비교

시퀀스 길이를 n , hidden dimension을 d 라고 할 때:

1) Self-Attention

- 복잡도: $O(n^2 \cdot d)$
- 모든 토큰이 모든 토큰을 참조하기 때문
- 시퀀스 길이가 길어질수록 비용 증가

하지만:

- 대부분 NLP task에서 $n < d$
 - 실제로는 RNN보다 효율적일 수 있음
-

2) Recurrent Layer

- 복잡도: $O(n \cdot d^2)$
 - hidden state를 매 step 계산해야 함
 - 순차적 계산 필수 → 병렬화 불가
-

3) Convolution Layer

- 복잡도: $O(k \cdot n \cdot d^2)$
(k = kernel size)
 - local receptive field만 처리
 - 병렬 가능하지만 long dependency는 비효율적
-

4.2 병렬화 가능성

- RNN:
 - 완전 순차적
 - 시간축 병렬화 불가
- CNN:
 - 병렬 가능
 - 하지만 깊은 layer 필요

- Self-Attention:
 - 모든 토큰 관계를 한 번에 계산
 - 완전 병렬화 가능
 - GPU 효율 매우 높음

→ 학습 속도 측면에서 큰 이점

4.3 Path Length (의존성 연결 거리)

가장 중요한 비교 포인트.

두 토큰 사이 정보가 전달되는 최소 연산 단계 수를 의미.

1) Self-Attention

- Path length = 1
 - 모든 토큰이 직접 연결됨
-

2) RNN

- Path length = $O(n)$
 - 토큰이 순차적으로 전달됨
 - 먼 거리일수록 gradient 전달 어려움
-

3) CNN

- Path length = $O(\log n)$ (dilated conv 사용 시)
 - 또는 $O(n/k)$
 - 여러 layer를 거쳐야 distant token 연결
-

4.4 핵심 결론

Self-Attention은:

- 장거리 의존성을 가장 직접적으로 모델링
- 병렬화 가능
- 깊은 네트워크 없이도 global dependency 학습 가능

논문에서 주장하는 핵심 메시지:

| Sequence modeling에서 recurrence는 구조적으로 필수적이지 않다.

5. Training

5.1 Training Data & Task

- 실험은 기계번역(machine translation) task에서 수행
 - 주요 데이터셋:
 - WMT 2014 English-German
 - WMT 2014 English-French
 - 데이터 전처리:
 - Byte-Pair Encoding (BPE) 사용
 - 단어를 subword 단위로 분해
 - vocabulary size 약 37K~32K 수준
-

5.2 Batch & Optimization

Optimizer

- Adam optimizer 사용
- $\beta_1 = 0.9$
- $\beta_2 = 0.98$
- $\epsilon = 10^{-9}$

기본 Adam과 β_2 값이 다름 ($0.999 \rightarrow 0.98$)

Learning Rate Schedule (중요)

Transformer는 고정 learning rate를 사용하지 않음.

Learning rate 공식:

$$lr = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup_steps}^{-1.5})$$

구성 의미:

1. 초반에는 warm-up 단계
 - step에 비례해 learning rate 증가
2. 이후에는 $\text{step}^{-0.5}$ 로 감소

기본 설정:

- warmup_steps = 4000

핵심 의도:

- 큰 모델에서 초기 학습 불안정성 방지
 - 안정적인 수렴 유도
-

5.3 Regularization

Dropout

- Dropout rate = 0.1
 - 적용 위치:
 - attention weights
 - feed-forward layers
 - positional encoding 합산 이후
-

Label Smoothing

- $\epsilon_{ls} = 0.1$
- 정답 분포를 one-hot이 아닌 부드러운 분포로 변환

이유:

- 과도한 확신(overconfidence) 방지
 - generalization 향상
-

5.4 Hardware & Training Time

- Base model:
 - 8개 NVIDIA P100 GPU
 - 약 12시간 학습

- Big model:
 - 더 큰 d_model, 더 많은 head
 - 약 3.5일 학습

병렬화 구조 덕분에:

- RNN 대비 학습 속도 매우 빠름

5.5 Inference

- Beam search 사용
- beam size = 4
- length penalty $\alpha = 0.6$

출력 길이 bias 문제를 보정하기 위해 length penalty 적용

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

6. Results

6.1 Machine Translation 성능

실험 데이터

- WMT 2014 English → German
- WMT 2014 English → French
- 평가 지표: **BLEU score**

6.2 주요 모델 비교

논문에서는 다음과 비교:

- RNN 기반 seq2seq
- ConvS2S (Convolutional Seq2Seq)
- GNMT (Google NMT)
- 기존 SOTA 모델들

6.3 English → German (WMT14 En-De)

Model	BLEU
기존 SOTA	약 26~27
Transformer (Base)	27.3
Transformer (Big)	28.4

Transformer Big 모델이 기존 최고 성능을 크게 상회.

6.4 English → French (WMT14 En-Fr)

Model	BLEU
기존 SOTA	40.4
Transformer (Big)	41.8

특히:

- 앙상블 없이 single model로 최고 성능 달성
- 훈련 시간도 더 짧음

6.5 Base vs Big 모델 차이

Base 모델

- $d_{\text{model}} = 512$
- 8 heads
- 6 layers
- 상대적으로 가벼움

- 빠른 학습

Big 모델

- $d_{\text{model}} = 1024$
 - 16 heads
 - 더 큰 feed-forward dimension
 - 더 높은 BLEU score
-

6.6 Attention 시각화 분석

논문은 attention weight를 시각화함.

관찰된 특징:

- 특정 head는 문법 관계를 학습
- 어떤 head는 long-distance dependency 학습
- 어떤 head는 positional alignment 담당

→ Multi-head attention이 서로 다른 관계를 분담해 학습한다는 증거

6.7 Self-Attention 거리 분석

논문은 attention이 얼마나 멀리 참조하는지 분석함.

결론:

- lower layers는 local context에 집중
- upper layers는 long-range dependency에 집중

→ 계층적 표현 학습 발생

7. Conclusion

7.1 핵심 기여

- Recurrence와 Convolution 없이
- Attention만으로 구성된 모델(Transformer) 제안
- Self-Attention 기반 구조가

- 병렬화 가능
 - 장거리 의존성 모델링에 유리
 - 학습 속도 빠름
-

7.2 실험적 검증

- WMT 2014 En-De / En-Fr에서 SOTA 달성
 - Single model로 기존 앙상블 모델보다 우수한 성능
 - 학습 시간 단축
 - 모델 구조 단순화
-

7.3 구조적 의미

논문의 핵심 주장:

Sequence modeling에서 recurrence는 필수 요소가 아니다.

Self-attention만으로도:

- 문맥 이해 가능
 - long-range dependency modeling 가능
 - 고성능 번역 모델 구현 가능
-

7.4 향후 확장 가능성

- Transformer는 번역에 국한되지 않음
 - 다른 NLP task에도 적용 가능
 - 더 큰 모델, 더 많은 데이터로 확장 가능
- 이후 BERT, GPT, ViT 등으로 발전